Classifying 100 Images on CIFAR-Dataset with 2D-CNN and EfficientNetB0

Nathanael Solagratia & Christian Simmons

Introduction

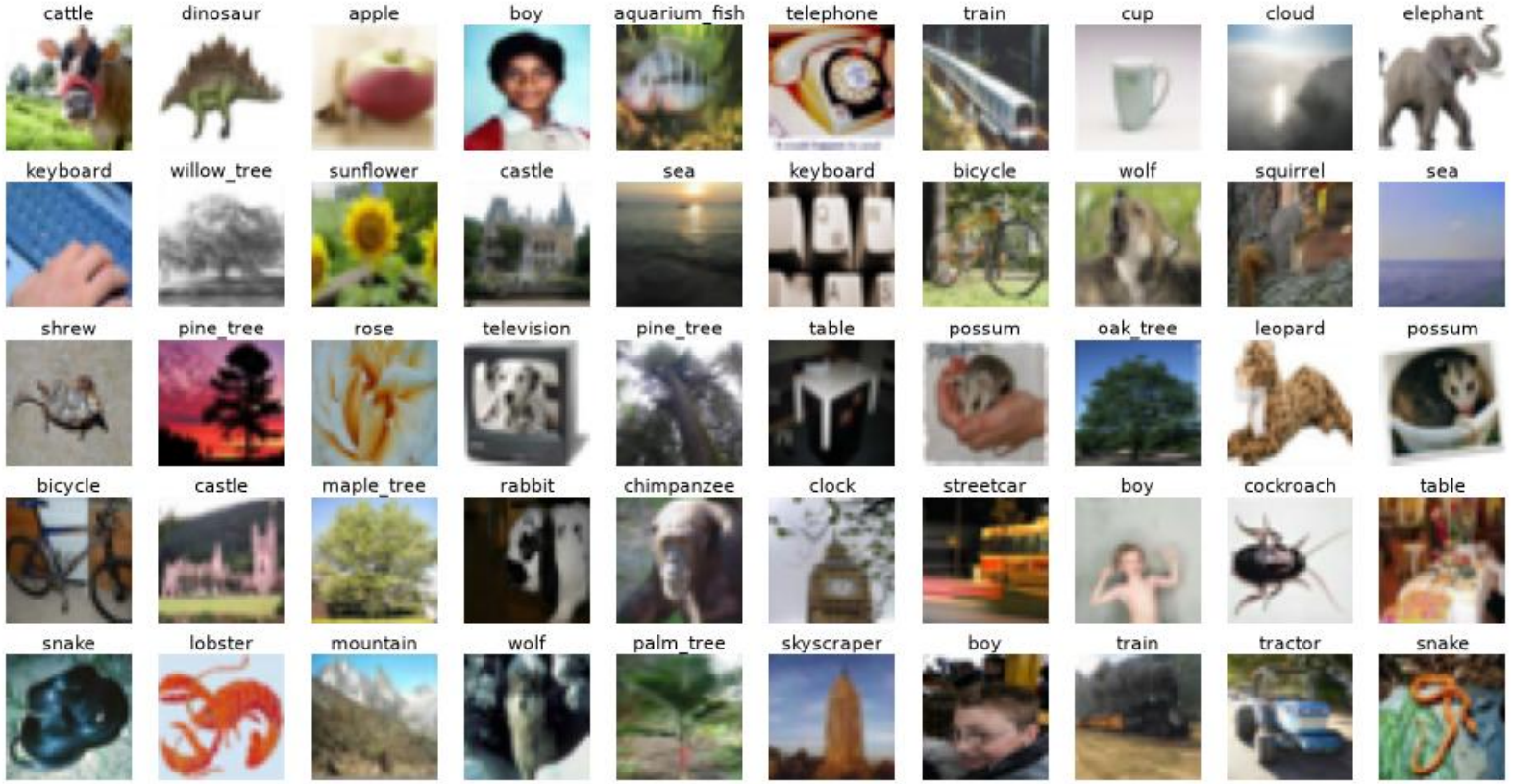
- Deep Neural Networks (DNNs) excel at complex tasks like image classification or recognition, where traditional machine learning (ML) reaches its limits.
- Introduced in 2009, the CIFAR-100 dataset remains a fundamental benchmark for evaluating fine-grained image recognition. The goal is to classify 100 distinct images, with only 32x32 pixel resolution.
- We chose this task because it clearly demonstrates how DNNs outperform classical ML as dataset size and class count scales larger and bigger in complex task such as recognizing 100 distinct pictures.

CIFAR-100 Dataset Overview

- Released in 2009 and it is widely used to train and evaluate image classification algorithms and deep neural networks.
- Contains 20 superclasses each with 5 distinct classes (Krizhevsky & Hinton, 2009).
- The entire dataset size is around 161 MB.

Superclass	Classes
aquatic mammals	beaver, dolphin, otter, seal, whale
fish	aquarium fish, flatfish, ray, shark, trout
flowers	orchids, poppies, roses, sunflowers, tulips
food containers	bottles, bowls, cans, cups, plates
fruit and vegetables	apples, mushrooms, oranges, pears, sweet peppers
household electrical devices	clock, computer keyboard, lamp, telephone, television
household furniture	bed, chair, couch, table, wardrobe
insects	bee, beetle, butterfly, caterpillar, cockroach
large carnivores	bear, leopard, lion, tiger, wolf
large man-made outdoor things	bridge, castle, house, road, skyscraper
large natural outdoor scenes	cloud, forest, mountain, plain, sea
large omnivores and herbivores	camel, cattle, chimpanzee, elephant, kangaroo
medium-sized mammals	fox, porcupine, possum, raccoon, skunk
non-insect invertebrates	crab, lobster, snail, spider, worm
people	baby, boy, girl, man, woman
reptiles	crocodile, dinosaur, lizard, snake, turtle
small mammals	hamster, mouse, rabbit, shrew, squirrel
trees	maple, oak, palm, pine, willow
vehicles 1	bicycle, bus, motorcycle, pickup truck, train
vehicles 2	lawn-mower, rocket, streetcar, tank, tractor

Example Images from CIFAR-100



- There are 500 training images,100 testing images per class, and 60K images in total (Krizhevsky & Hinton, 2009).
- Each image resolution is 32x32 pixels.

Data Preparation

- The entire CIFAR-100 dataset consists of 60K images, and we split the dataset into 40K training, 10K validation and testing (Krizhevsky & Hinton, 2009).
- Entire dataset transferred to GPU memory for zero data-transfer latency. GPU used: RTX 4060.
- Training data augmentations (random cropping & horizontal flipping) executed directly on the GPU.
- For hyperparameter tuning, we used Optuna framework to find the optimal learning rate, batch size, weight decay, and dropout rate to increase performance (Akiba et al., 2019).

2D-CNN Model Description

- Built with sequential blocks of stacked 3x3 convolutions, doubling the feature channels at each step while halving the image resolution.
- For regularization, we used Batch Normalization and GELU (Gaussian Error Linear Unit) activations instead of ReLU.
- Uses Global Adaptive Pooling instead of massive flattened Linear layers, to make the training faster.

```python
class Improved2DCNN(nn.Module):
    def __init__(self, dropout_rate=0.3, num_classes=15):
        super().__init__()

        # Feature Extractor: 4 Blocks with Stacked Convolutions
        self.features = nn.Sequential(
            # Block 1: 32x32 -> 16x16
            nn.Conv2d(3, 64, kernel_size=3, padding=1), nn.BatchNorm2d(64), nn.GELU(),
            nn.Conv2d(64, 64, kernel_size=3, padding=1), nn.BatchNorm2d(64), nn.GELU(),
            nn.MaxPool2d(2, 2),
            nn.Dropout2d(dropout_rate * 0.5),

            # Block 2: 16x16 -> 8x8
            nn.Conv2d(64, 128, kernel_size=3, padding=1), nn.BatchNorm2d(128), nn.GELU(),
            nn.Conv2d(128, 128, kernel_size=3, padding=1), nn.BatchNorm2d(128), nn.GELU(),
            nn.MaxPool2d(2, 2),
            nn.Dropout2d(dropout_rate * 0.5),

            # Block 3: 8x8 -> 4x4
            nn.Conv2d(128, 256, kernel_size=3, padding=1), nn.BatchNorm2d(256), nn.GELU(),
            nn.Conv2d(256, 256, kernel_size=3, padding=1), nn.BatchNorm2d(256), nn.GELU(),
            nn.MaxPool2d(2, 2),
            nn.Dropout2d(dropout_rate * 0.5),

            # Block 4: 4x4 -> 2x2
            nn.Conv2d(256, 512, kernel_size=3, padding=1), nn.BatchNorm2d(512), nn.GELU(),
            nn.Conv2d(512, 512, kernel_size=3, padding=1), nn.BatchNorm2d(512), nn.GELU(),
            nn.MaxPool2d(2, 2)
        )

        # Adaptive Pooling: Force feature maps to (1, 1) regardless of input resolution
        self.global_pool = nn.AdaptiveAvgPool2d((1, 1))

        # Classifier Head (512 inputs instead of 4096)
        self.classifier = nn.Sequential(
            nn.Dropout(dropout_rate),
            nn.Linear(512, 256), nn.GELU(), nn.Dropout(dropout_rate),
            nn.Linear(256, num_classes)
        )

    def forward(self, x):
        x = self.features(x)
        x = self.global_pool(x)
        x = torch.flatten(x, 1)
        return self.classifier(x)
```

EfficientNetB0 Model Description

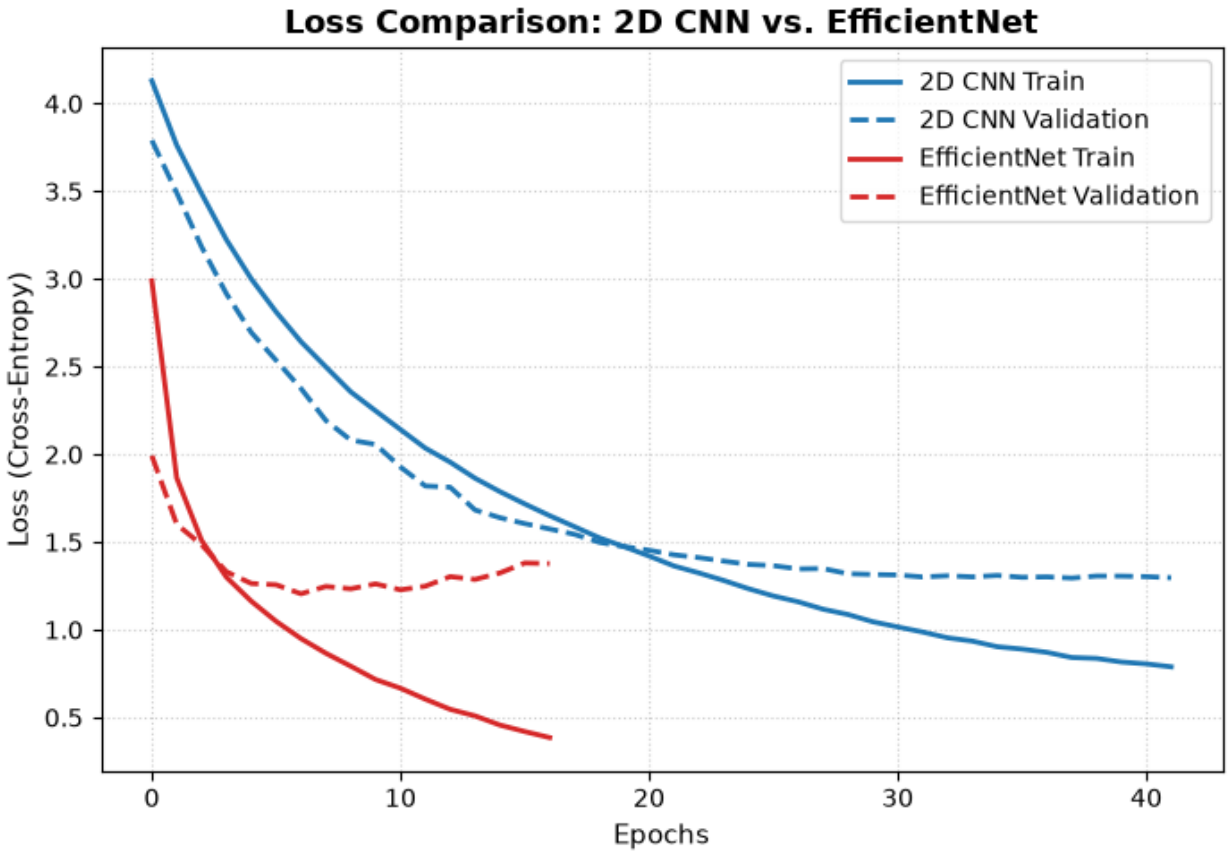
```python
def make_efficientnet(dropout_rate=0.2, num_classes=100):
    model = models.efficientnet_b0(weights=models.EfficientNet_B0_Weights.DEFAULT)
    model.features[0][0] = nn.Conv2d(3, 32, kernel_size=3,
                                     stride=1, padding=1, bias=False)

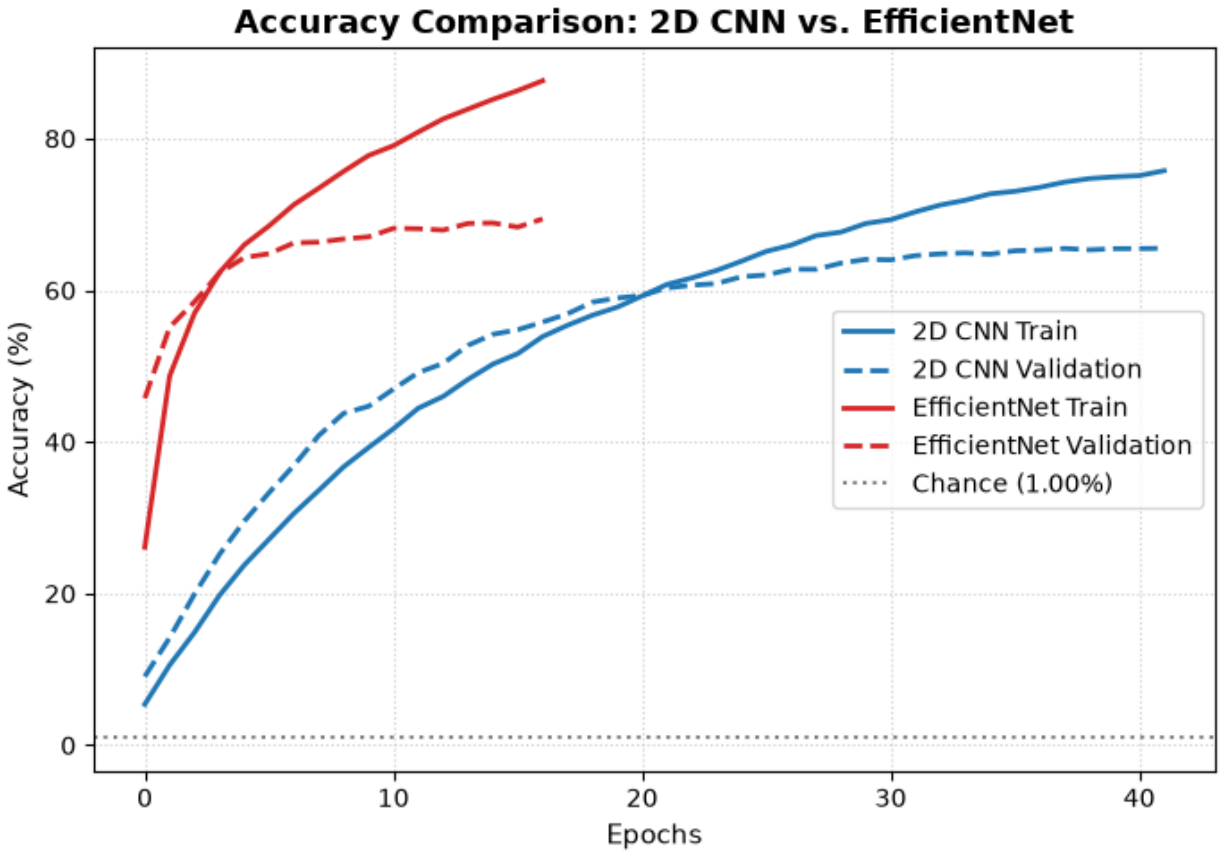
    # Overwrite the default PyTorch Dropout layer (model.classifier[0])
    model.classifier[0] = nn.Dropout(p=dropout_rate, inplace=True)
    model.classifier[1] = nn.Linear(model.classifier[1].in_features, num_classes)

    return model.to(device)
```

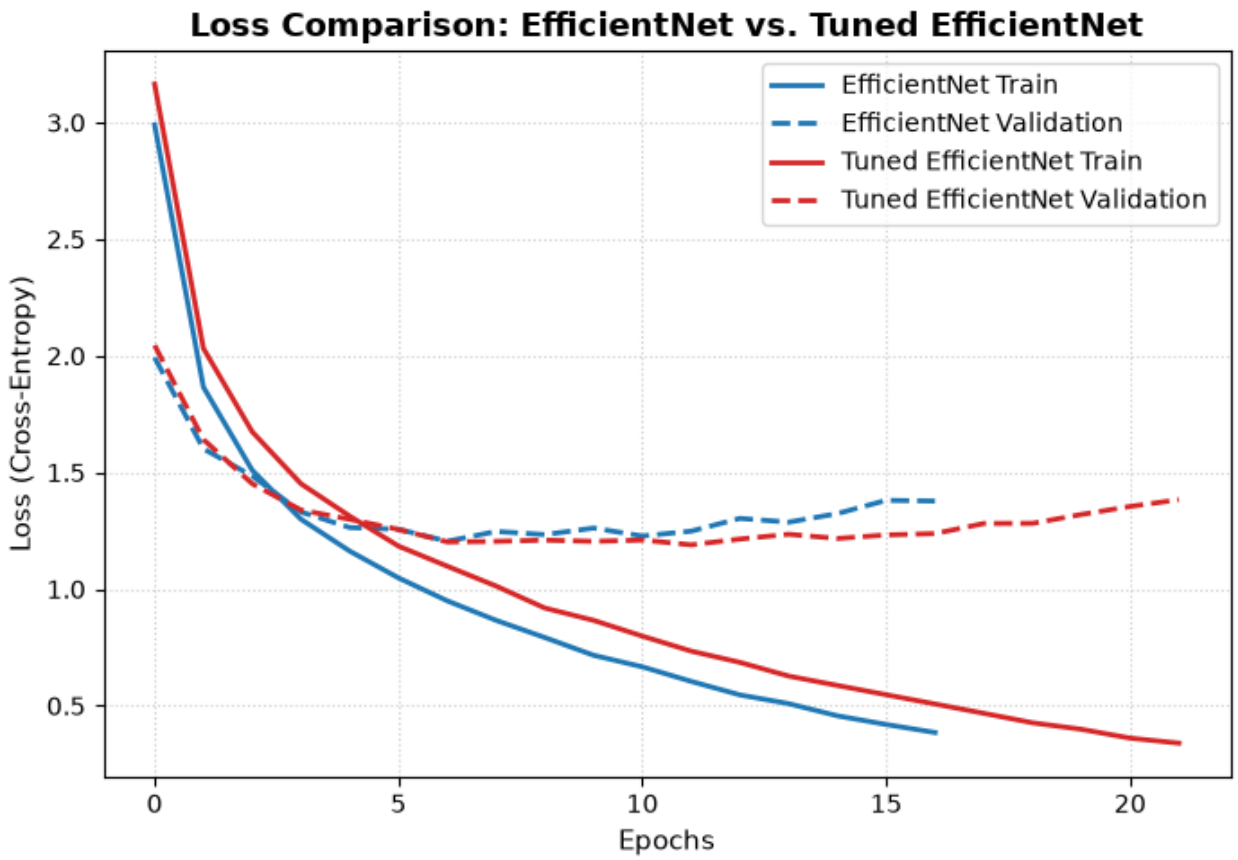
- Uses ImageNet-1k weights (1.28M images) for faster convergence and higher accuracy on CIFAR-100 (Tan & Le, 2019).
- Mobile Inverted Bottlenecks (MBConv) is a lightweight depthwise convolutions that reduces computation without sacrificing feature quality (Tan & Le, 2019).
- Replaced the default 1,000-class output with Dropout and a 100-class Linear layer to make it work with CIFAR-100.

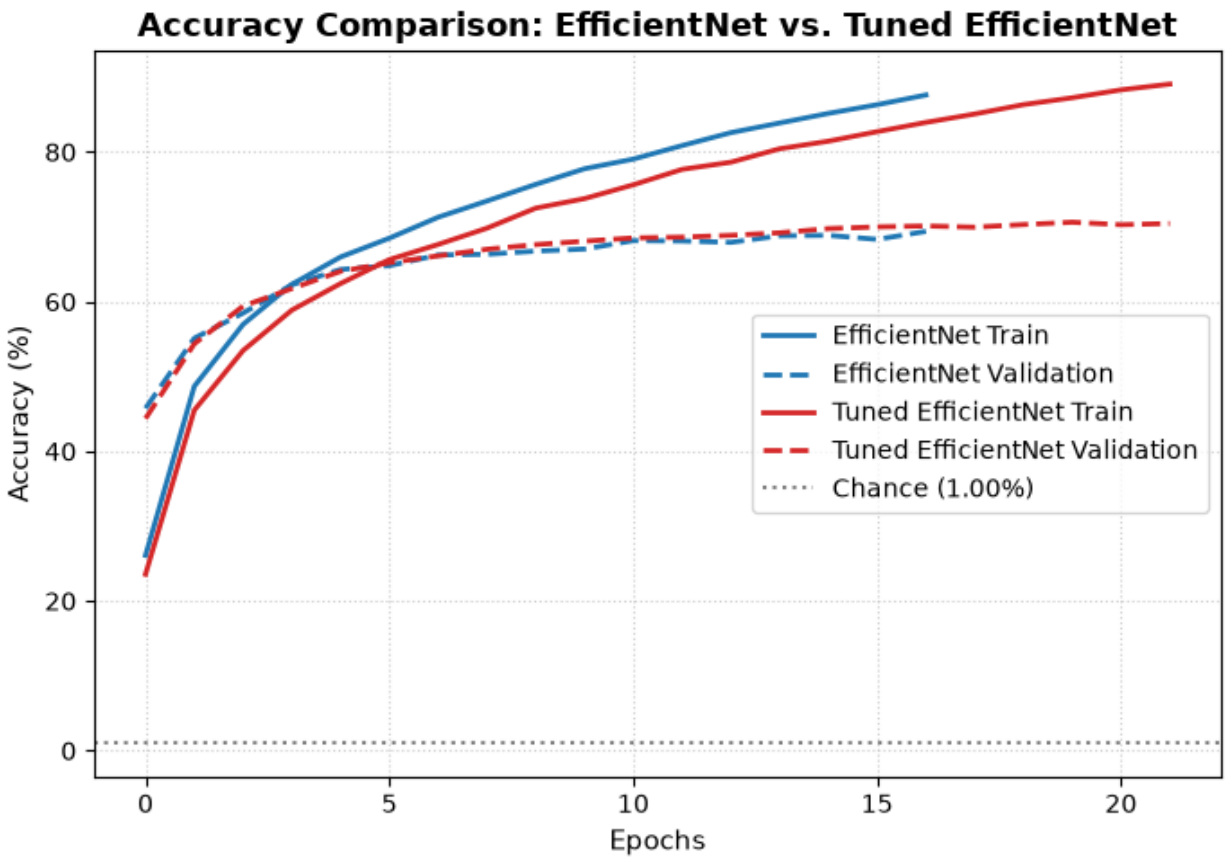
2D CNN & EfficientNetB0 Loss & Accuracy



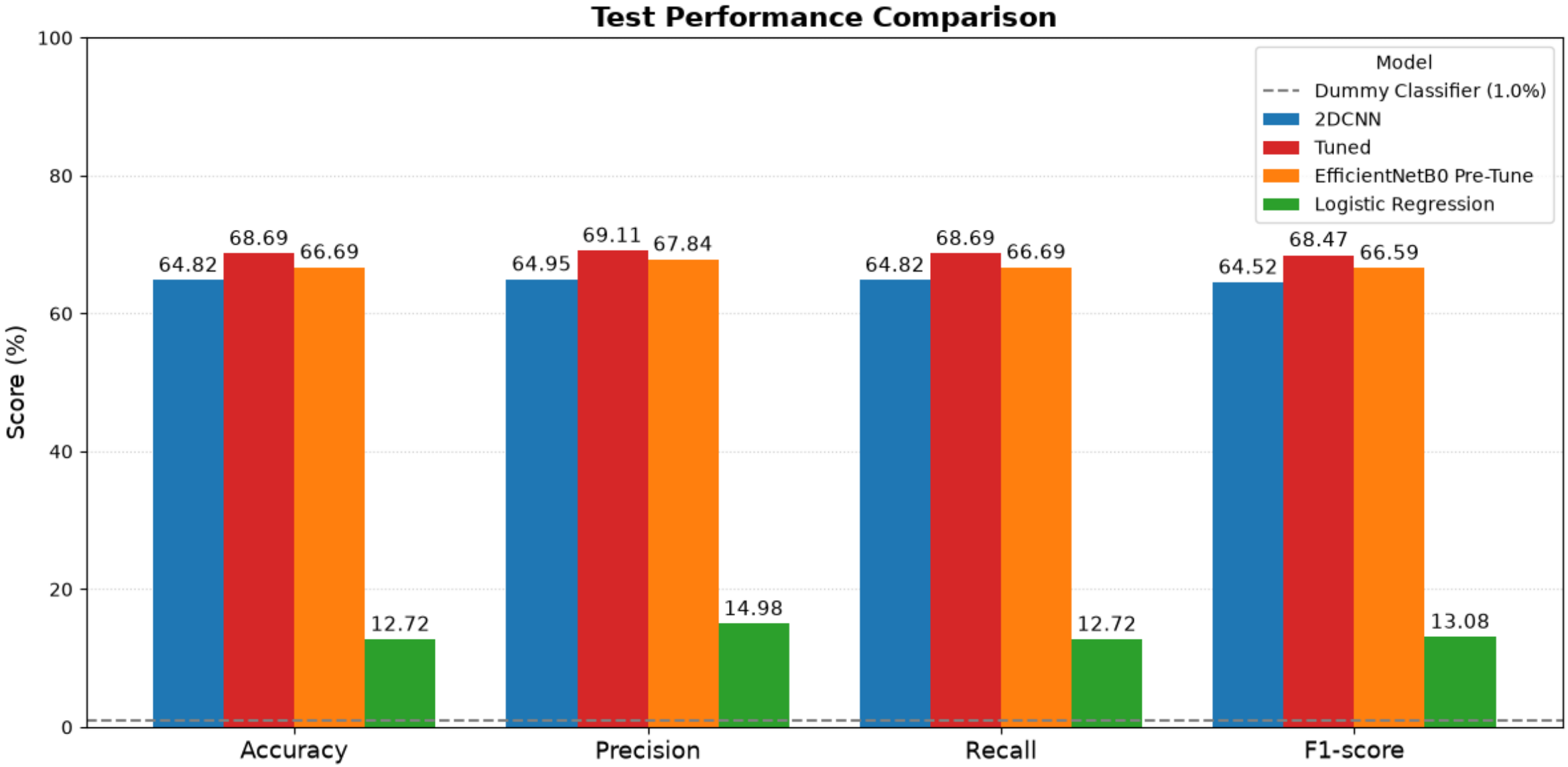


Tuned EfficientNetB0 Loss & Accuracy

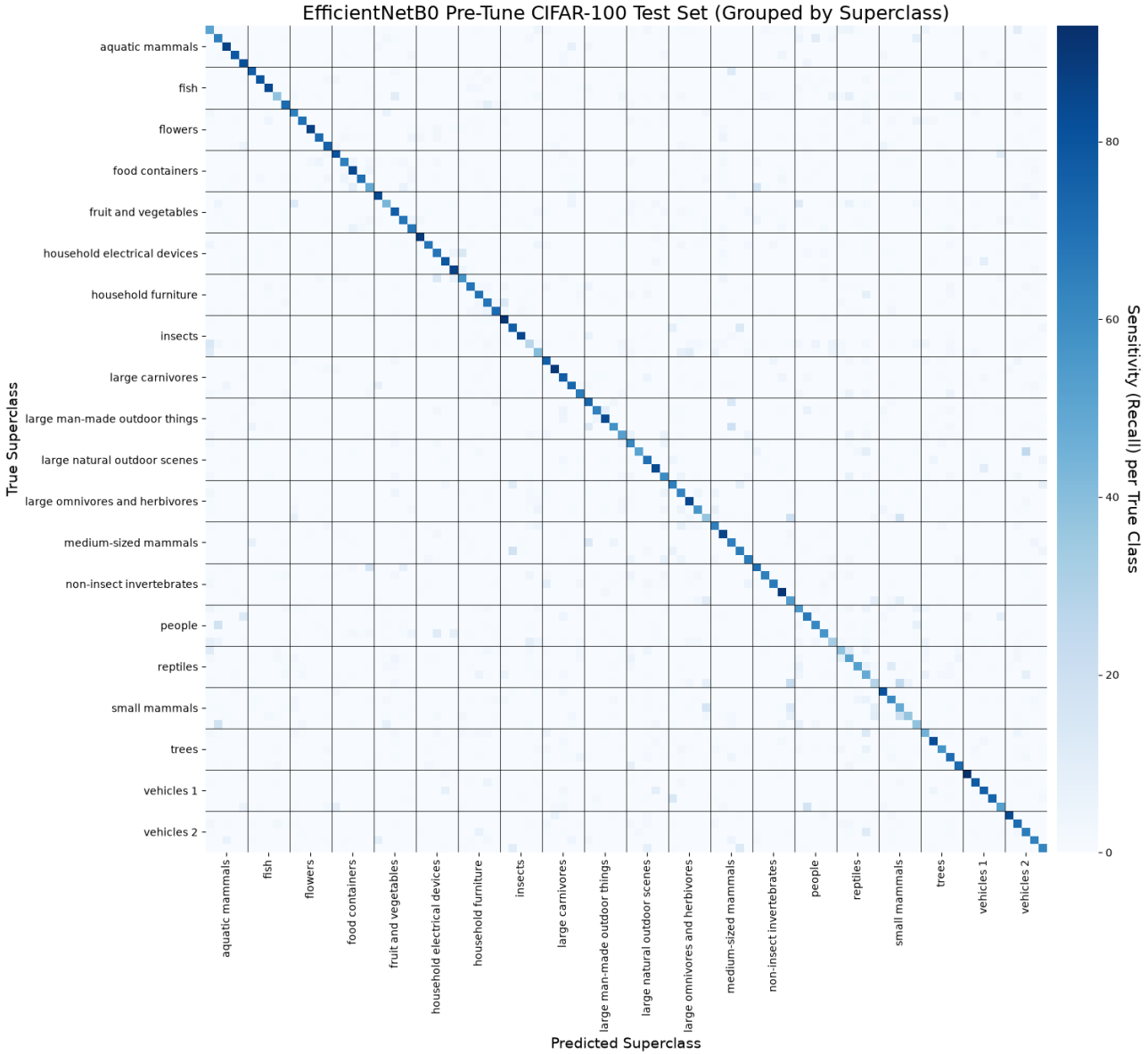


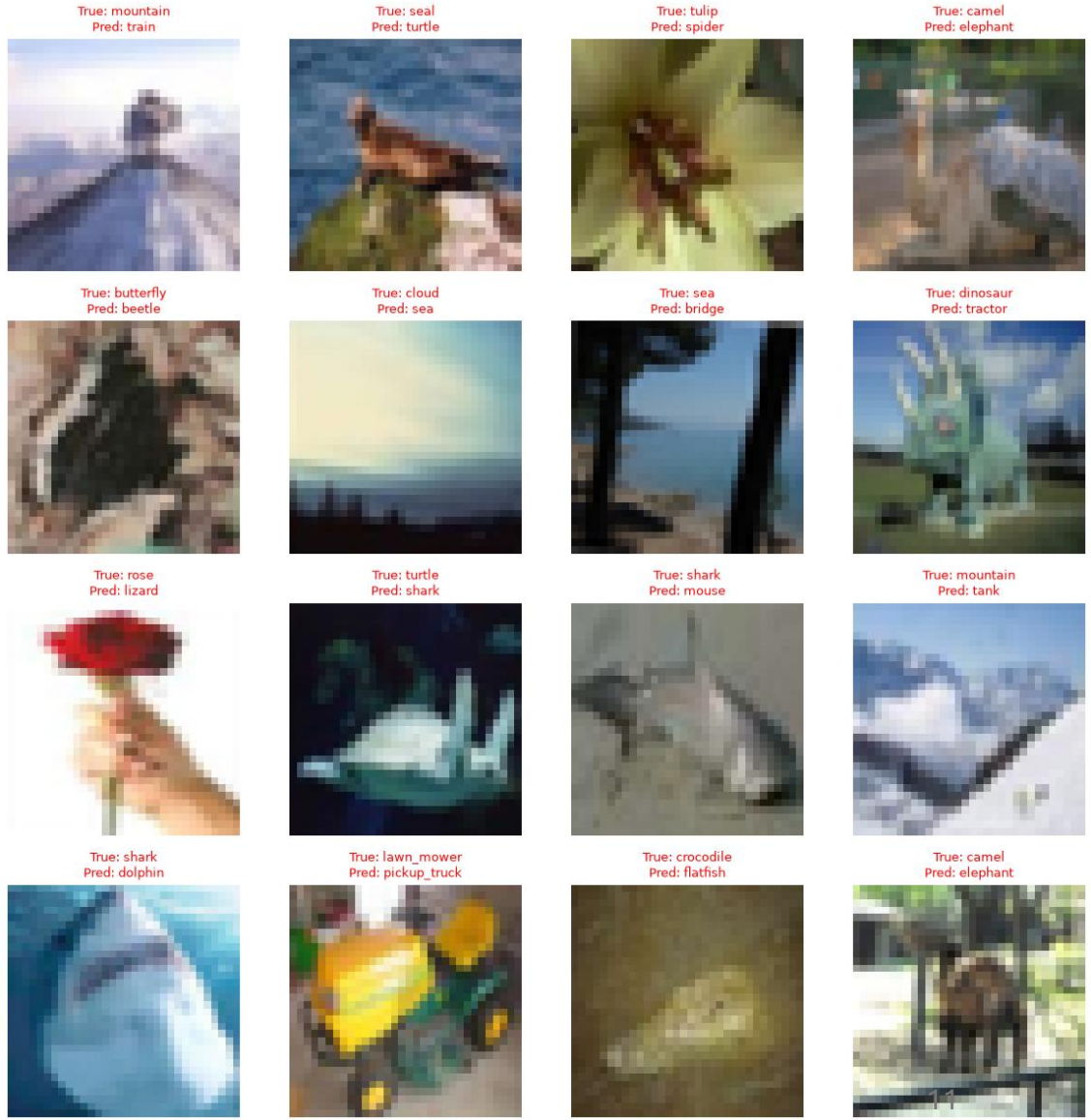


Performance Metrics

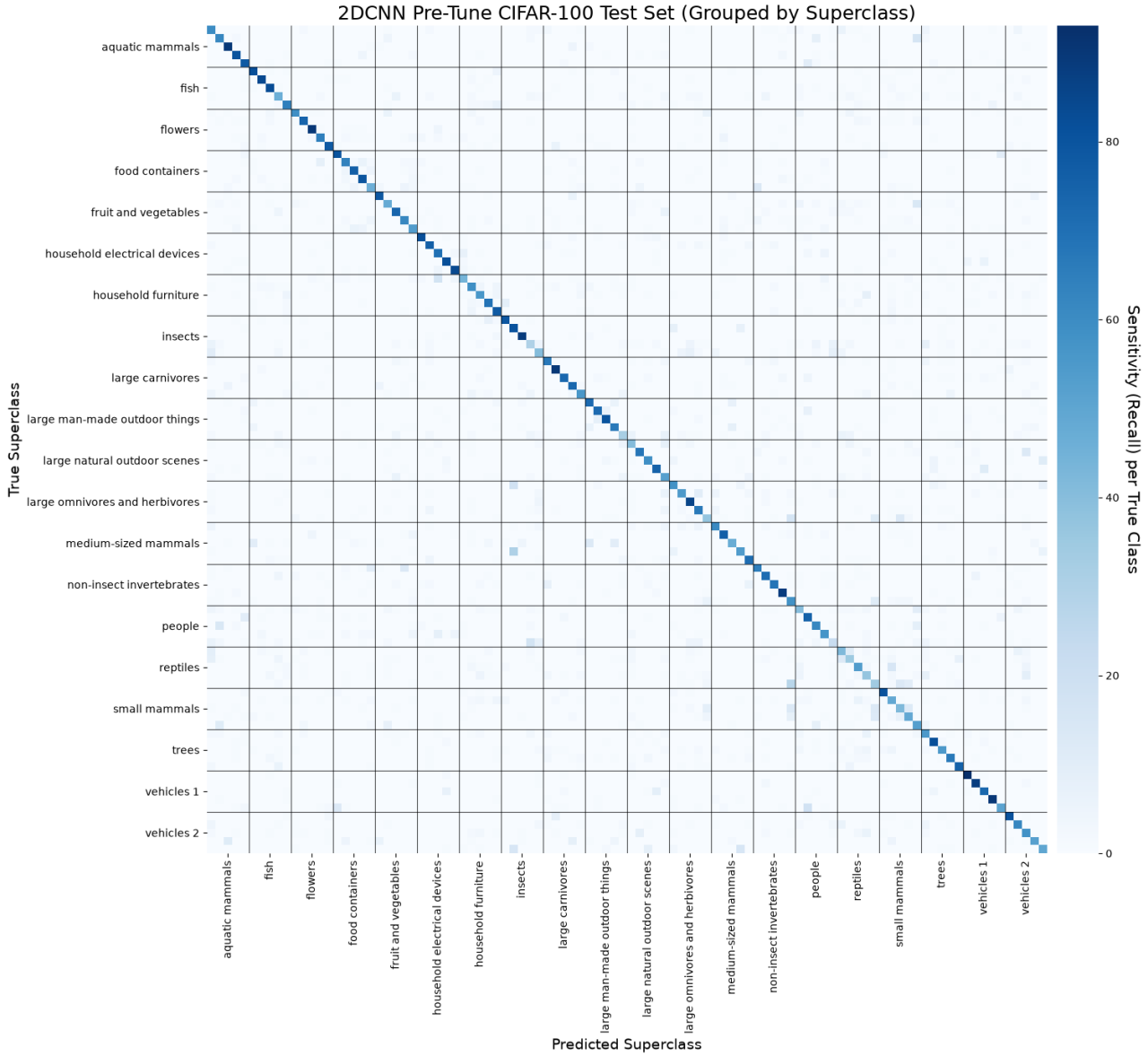


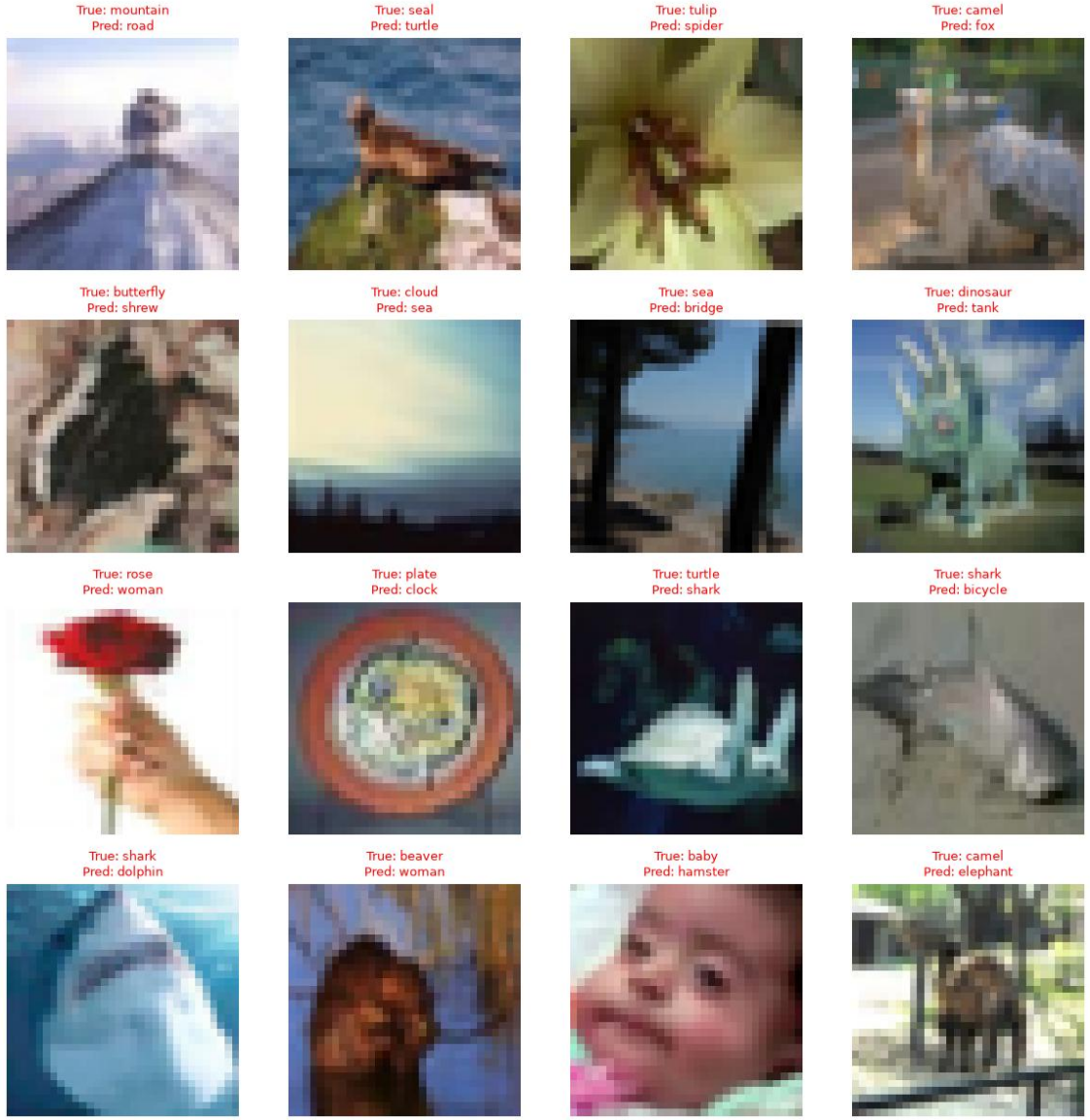
EfficientNetB0 Error Analysis



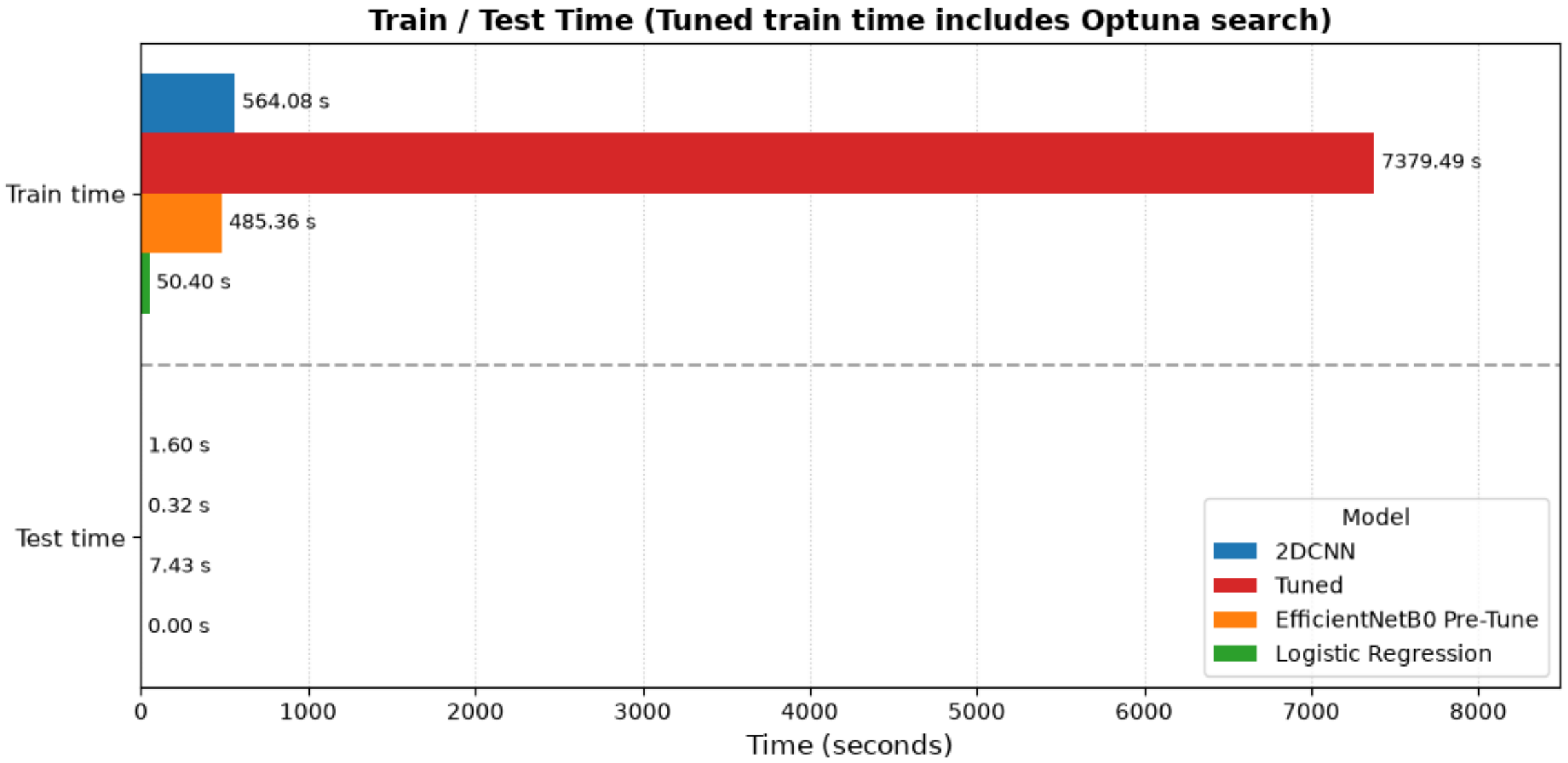


2D CNN Error Analysis





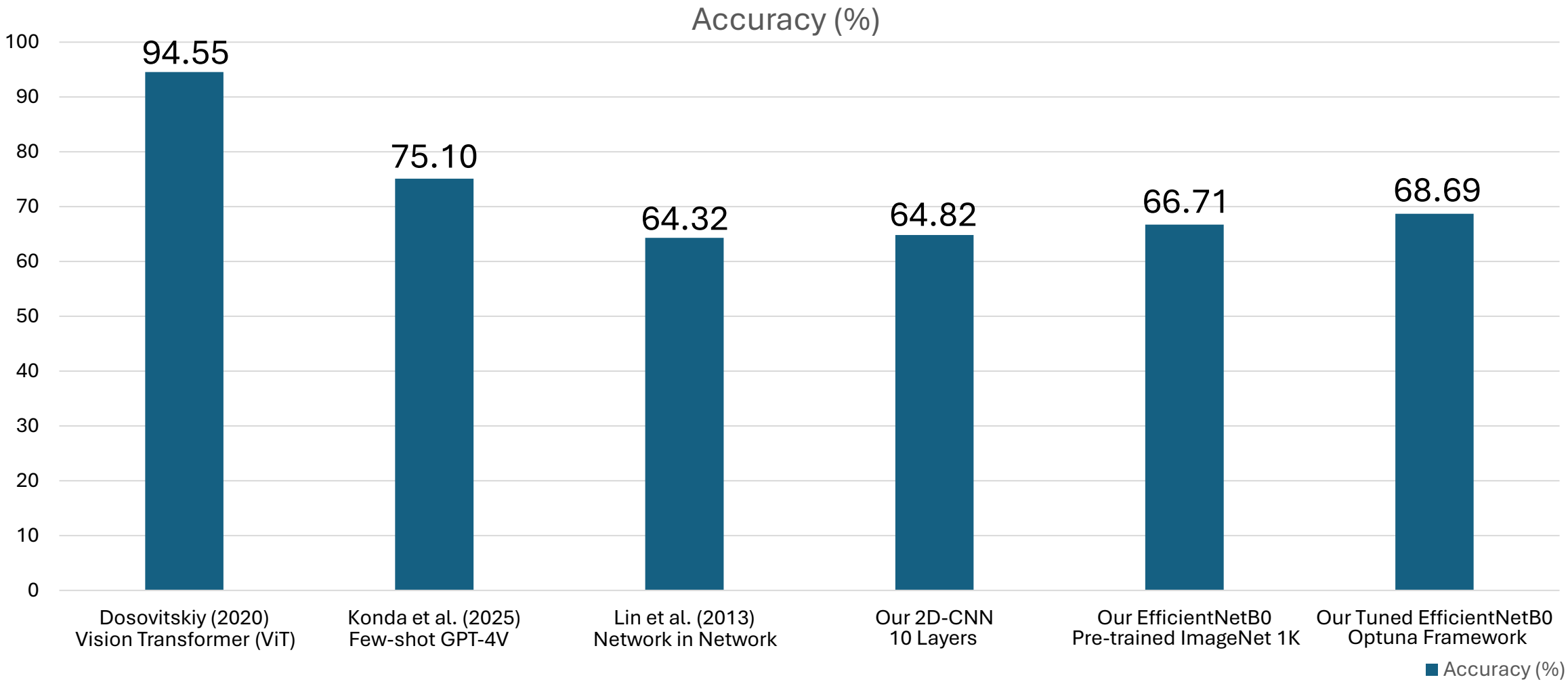
Evaluation Time



Parameters Used

- Tuned EfficientNetB0:
  Learning rate = 0.0005134369783823349
  Dropout rate = 0.4
  Weight decay = 0.00010607190802583915
  Batch size = 32
- Base EfficientNetB0 and 2D-CNN:
  Learning rate = 0.001
  Dropout rate = 0.3
  Weight decay = 0.0001
  Batch size = 64
- All models used 50 epochs with early stopping.

Comparison Against Previous Studies



Conclusion

- **Baseline, custom 2D-CNN Holds Its Own:** The custom 10-layer 2D-CNN accuracy (64.82%) performed almost on par with the pre-trained EfficientNetB0 baseline (66.71%).
- **Architecture is the Limiting Factor:** Parameter optimization is not the primary limiting factor. Optuna tuning cost 15x more compute time (7,379s vs. 485s) for a meager ~2% accuracy gain (68.69%). With this setup, CNN is not going to achieve Vision Transformer level performance of 94.55%.
- **Shared Failures:** Both models hit the same classification wall, consistently confusing visually similar classes, such as mistaking camels for elephants or sharks for dolphins and turtle.

References

- Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019, July). Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining* (pp. 2623-2631).
- Dosovitskiy, A. (2020). An image is worth 16x16 words: Transformers for image recognition at scale. arXiv preprint arXiv:2010.11929.
- Konda, R., Syed, S. A., Yadav, H., Joodala, A., Mungara, P., & Katta, S. K. R. (2025, October). Performance comparison of deep learning, classical ML, and large language models in scalable image classification tasks. In 2025 7th International Conference on Innovative Data Communication Technologies and Application (ICIDCA) (pp. 1410-1415). IEEE.
- Krizhevsky, A., & Hinton, G. (2009). Learning multiple layers of features from tiny images.
- Lin, M., Chen, Q., & Yan, S. (2013). Network in network. arXiv preprint arXiv:1312.4400.
- Tan, M., & Le, Q. (2019, May). Efficientnet: Rethinking model scaling for convolutional neural networks. In International conference on machine learning (pp. 6105-6114). PmLR.